COS214: TIVIDY

Ethan Van Eyden (u25260244)
Christian Nunes (u25138309)
Stefan van Staden (u25031784)
Lesego Tebeile (u25143230)

Table of Contents

Task 1: Research 2
Task 2: Scenario definition and Scenario 4
Task 3: UML activity diagrams 6
Task 4: Design Patterns 9
Participant mapping 9
Task 5: UML Class diagram 13
Task 6: Runtime Behaviour Diagrams 14
Task 7: Design Decisions and Revisions 16

Task 1: Research

A workflow is a computerized representation of a business process. It lists mandatory activities, their order, conditions, necessary data, users, and needed tools. Now a Workflow Management System is a system that is used to create, execute and monitor workflows. It coordinates the people and activities involved in a business process.

A workflow's definition differs from its instance as a definition describes how the process should work whereas an instance is one actual execution of the process, allowing for many instances of the same process to run at the same time and at different stages.

A workflow is made up of individual activities. Each activity can specify what needs to be done, who performs it, what tools are needed, and how it connects to other activities.

Work is assigned to users based on roles and staff assignments. The workflow system determines where an activity should be performed and can route work between different users.

Workflows need to support different paths depending on decisions or outcomes. If an activity cannot continue normally, the workflow should be able to follow an alternative path or compensate for previous work.

Workflows can involve both **people and computer programs**. The system needs information about roles, staff assignments, and organizational structures so that work can be given to the correct participants.

A WFMS must handle failures so that information is not lost and the entire process does not stop. It may need to reroute work to another available component. A WFMS needs to coordinate and integrate different applications, databases, and other technologies within the same workflow. This is especially important in large organizations with different systems and platforms.

The workflow definition should be separate from its individual instances. This allows the same definition to be reused for many different executions. This makes the system suitable for different business processes rather than only one specific process.

Workflow processes cannot predict every possible exception. Therefore, the system needs flexible navigation so that different paths can be selected when unexpected situations occur rather than having every situation permanently hard-coded.

References

1. Gertrude Kappel, Peter Lang, Stefan Rausch-Schott & Werner Retschitzegger (1995), Workflow Management Based on Objects, Rules, and Roles, IEEE Bulletin of the Technical Committee on Data Engineering, Vol. 18(1), pp. 10–17.

http://sites.computer.org/debull/95MAR-CD.pdf#page=20

2. Georgakopoulos, D., Hornick, M. & Sheth, A. (1995). An Overview of Workflow Management: From Process Modeling to Workflow Automation Infrastructure. Distributed and Parallel Databases, 3, 119–153. DOI: 10.1007/BF01277643.

https://link.springer.com/content/pdf/10.1007/bf01277643.pdf

3. Schulz, K.A. & Orlowska, M.E. (2004). Facilitating cross-organisational workflows with a workflow view approach. Data & Knowledge Engineering, 51, 109–147.

https://doi.org/10.1016/j.datak.2004.03.008

4. Kumar, A., van der Aalst, W.M.P. & Verbeek, E.M.W. (2002). Dynamic work distribution in workflow management systems: How to balance quality and performance. Journal of Management Information Systems, 18(3), 157–193. DOI: 10.1080/07421222.2002.11045693.

https://www.tandfonline.com/doi/pdf/10.1080/07421222.2002.11045693

Task 2: Scenario definition and Scenario

Initial scope of TIVIDY:

TIVIDY is a generic workflow management system. It stores definitions, creates independent running instances from them, and coordinates the work, participants, rules and events needed to carry each instance outcome.

Main responsibilities:

Building and validating definitions; creating and starting instances; representing work as nested groups and individual items; assigning work using configurable rules; managing the lifecycle of work and the phases; routing results through approval or rejecting and/or escalations; react to internal and external events and handle according to intent; record history and have a rollback method.

Boundaries:

TIVIDY decides who does the work, in what order and what happens with the result of said work. It does not perform the work itself, it does not own external systems- only talks to them through external connectors, and it does not hard code any one organization's rules.

2.2 Scenario: Assignment Marking in a University Module

Environment: a university Computer Science department running a large module with hundreds of students, marked by several assistant lecturers (ALs).

Process: the full life of one assignment: defining it, collecting submissions, marking in parallel, moderating, approving and releasing marks.

Participants: module coordinator (defines the assignment, final review, approves release); lecturer (reviews after a first rejection); assistant lecturers (mark and re-mark); students (submit and receive marks).

Major work: define the assignment, collect submissions, mark each submission question by question, review marks, re-mark rejected scripts, approve, release marks. Structure: module → assignment → student submission → questions.

Decisions and routing: Was the submission on time? Which AL is assigned? Are marks accepted or rejected? Who reviews next (lecturer after 1 rejection, coordinator after 3)?

Was the marking deadline missed (escalate)? Is everything accepted so the phase can advance?

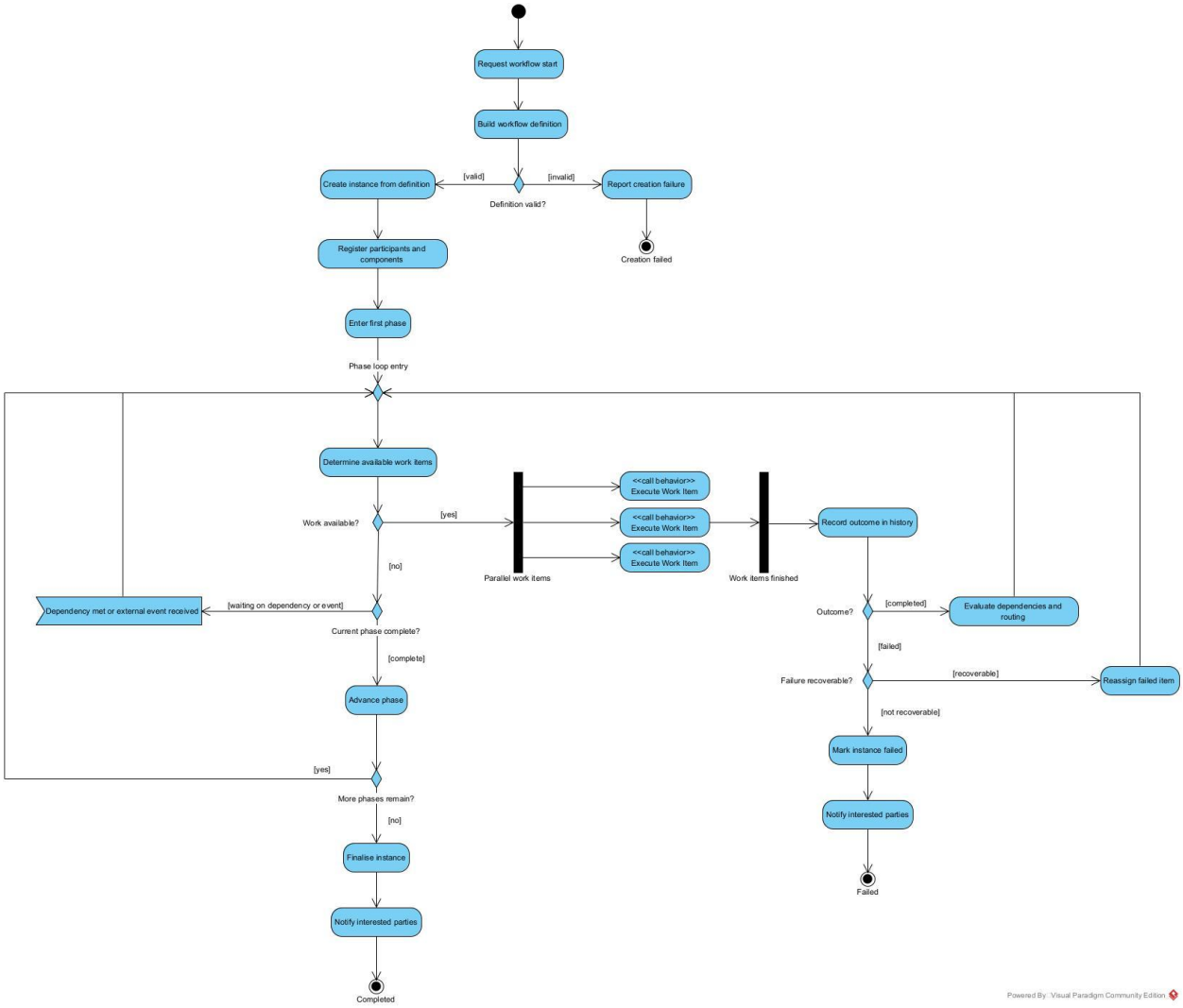
Approvals and dependencies: marking starts only after collection closes; review only after marking; release only after all scripts are accepted and the coordinator approves.

Failures and exceptions: late or missing submissions; an AL missing the deadline or being unavailable; repeated rejection; the LMS being unreachable at release (retry, then fail); invalid phase changes (e.g. releasing before approval) being refused.

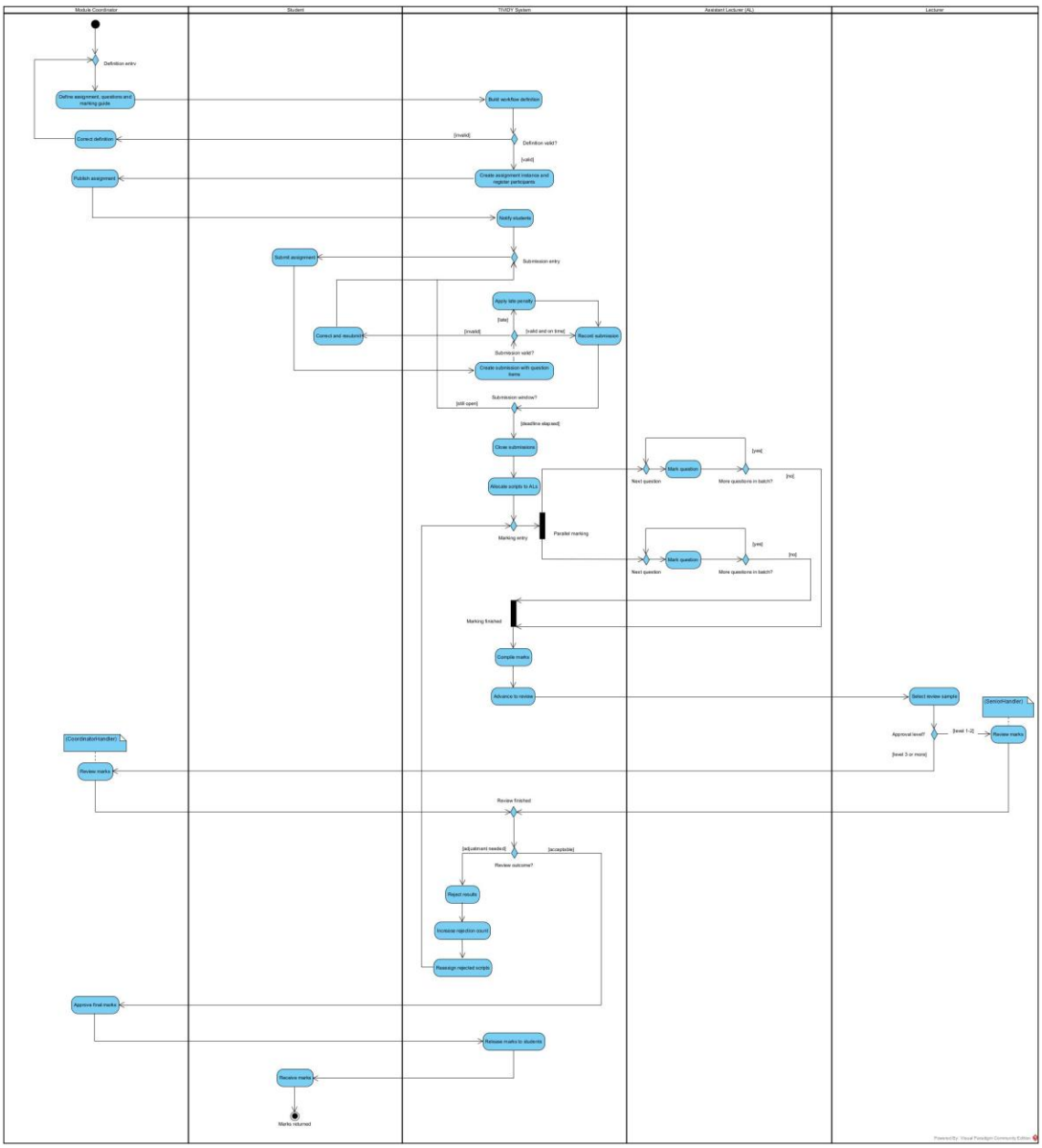
External systems: an LMS (submission intake, mark release) and an email service (notifications).

Expected outcome: all submissions are marked, reviewed and approved, marks are released to students, and a full audit history shows who did what and when. Items that cannot be completed end as rejected or failed and are flagged for follow-up.

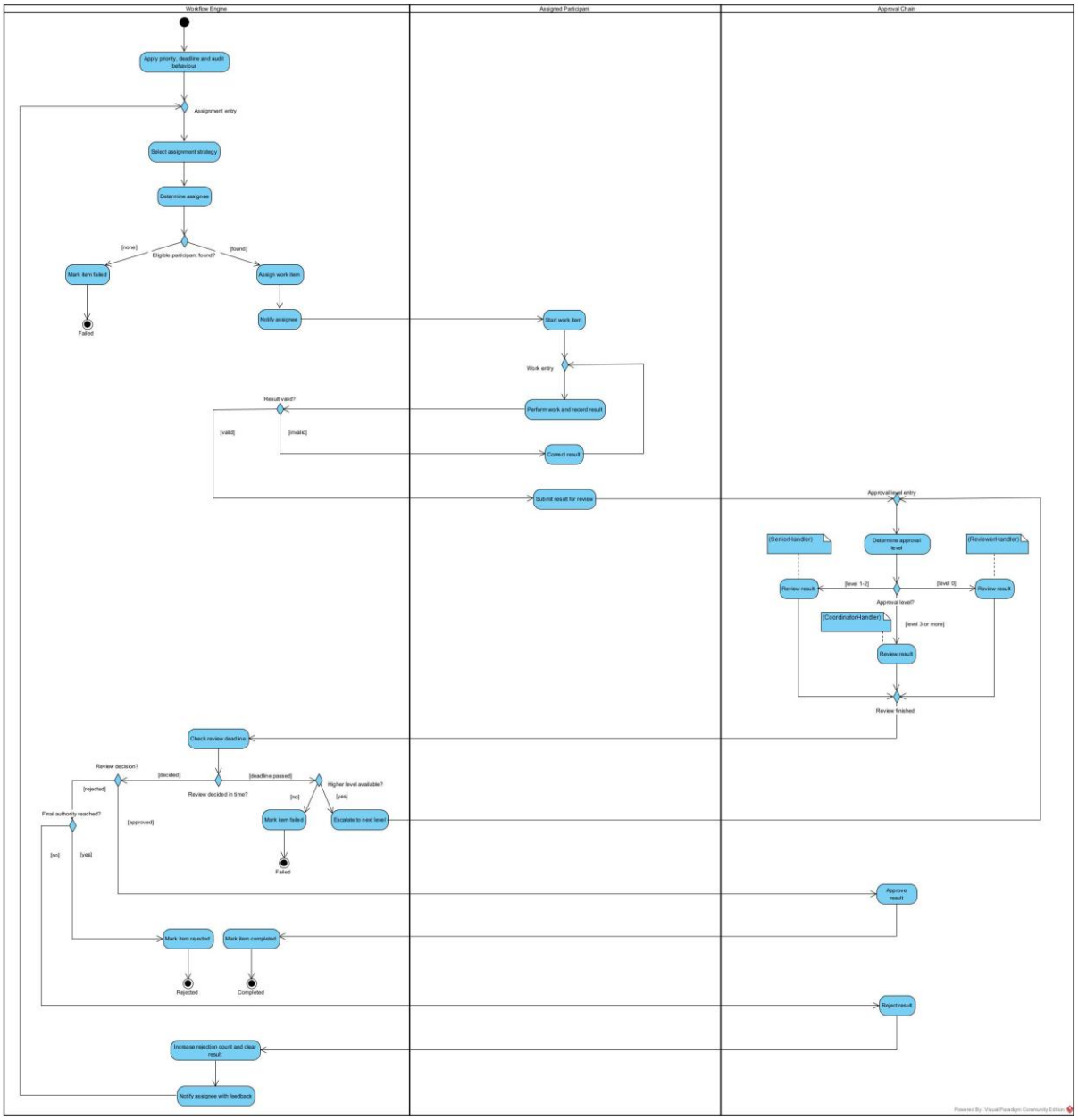
Task 3: UML activity diagrams



Shows the life of one running workflow instance. A workflow definition is built and validated, an instance is created from it, participants are registered and the first phase begins. In each phase the engine finds available work items, runs them in parallel through the called activity Execute Work Item (Diagram 2), records the outcome and evaluates dependencies. Failures are retried or end the instance, external events can resume a waiting phase, and the instance finishes after the last phase.



Applies TIVIDY to assignment marking, from creation to returning marks. The module coordinator defines the assignment, the system builds the workflow and students submit before the deadline. Assistant lecturers then mark scripts in parallel. The lecturer reviews the marks after a first rejection, and the module coordinator after repeated rejections. Rejected scripts are reassigned for re-marking. Once the marks are accepted, the coordinator approves them and the system releases them to students.



Details one work item from assignment to a final outcome, across three swimlanes. The engine applies priority, deadline and audit behaviour, selects an assignment strategy and assigns the item. The participant performs and corrects the work, then submits it. An approval chain reviews the result at a level set by the rejection count. Missed deadlines escalate to the next level. The item ends as completed, rejected or failed, and a rejection returns it for reassignment.

Task 4: Design Patterns

Participant mapping

Patterns:

(Main idea)

Composite: (Map module, assignment and questions) in general it models any body of work that is broken into groups of items, such as submissions, documents or inspections.

Component: WorkComponent

Composite: WorkContainer

Leaf: Workitem

COS 214 (WorkContainer, the "module")

├── Assignment 1 (WorkContainer)

│ ├── Student A submission (WorkContainer)

│ │ ├── Q1 (WorkItem)

│ │ └── Q2 (WorkItem)

│ └── Student B submission (WorkContainer)

├── Test 1 (WorkContainer)

└── Practical 6 (WorkContainer)

State:

Context: WorkContainer

State: WorkFlowPhase

Concrete State:

CollectingState, ProcessingState, ReviewingState, PublishingState

CollectingState, (Submissions)

ProcessingState (Marking)

ReviewingState (Moderating)

PublishingState (release marks)

transitions: Collecting → Processing → Reviewing → Publishing, (Reviewing → Processing when rejected.)

State:

Context: WorkItem

State: WorkItemState

ConcreteState: AssignedState, CancelledState, CreatedState, EscalatedState, InProgressState, RejectedState

Strategy: (When container enters Processing, we decide which strategy to use)

Context: AssignmentEngine

Abstract Strategy: AssignmentStrategy

Concrete Strategy: RoleBased, Workload, RoundRobin

Decorator:

Component: WorkComponent

ConcreteComponent: WorkItem, WorkContainer

Decorator: WorkDecorator

ConcreteDecorator: PriorityDecorator, AuditDecorator, DeadlineDecorator

PriorityDecorator(Raises urgency on task)

AuditDecorator(Record who performs an action)

DeadlineDecorator(Checks if item is completed in time)

Command: WorkCommand

ConcreteCommand: AssignItem, RecordResult, ApproveResult, RejectResult, AdvancePhase

AssignItemCommand: Asks the assignment strategy for a participant, then sets that participant as the item's assignee

RecordResultCommand: Saves a mark or result on a `WorkItem`

ApproveResultsCommand: The reviewer approves the results, so the container moves from Reviewing to Publishing

RejectResultsCommand: The reviewer rejects the results, so the container moves from Reviewing back to Processing and the rejected items are cleared for reassignment

AdvancePhaseCommand: Tells a container to move to its next phase (Collecting → Processing, and so on)

Invoker: ActionHistory

Receiver: WorkItem, WorkContainer

Iterator:

Iterator: WorkIterator

ConcreteIterator: DepthFirstIterator, PriorityIterator(urgent or important items first)

Aggregate: Workcomponent

ConcreteAggregate: WorkContainer

Mediator: (Coordinate change like assign items, notify, advance states)

Mediator: WorkflowMediator

Colleague: Colleague

ConcreteMediator: WorkflowCoordinator

ConcreteColleagues:

WorkContainer (Asks to advance states)

WorkItem (Reports when result is recorded)

AssignmentEngine (Mediator tell when to assign strategy to items)

ActionHistory (Records actions when told to by mediator)

Helper: Workregistry

Observer:

Subject:EventBus

ConcreteSubject: EventBus

Observer: Observer

ConcreteObserver: LMSObserver, EmailObserver

Event: WorkflowEvent

Builder: Create workflow(Builds Composite)

Builder: WorkflowBuilder

Director: WorkflowDirector

ConcreteBuilder: AssignmentMarkingBuidler

Product: WorkflowDefinition

Helper: WorkRegistry

Chain of Responsibility:

Handler: ApprovalHandler

Request: ApprovalRequest

ConcreteHandler: ReviewHandler(example AL does marking)

, SeniorHandler(lecturer marks example when item get rejected once),

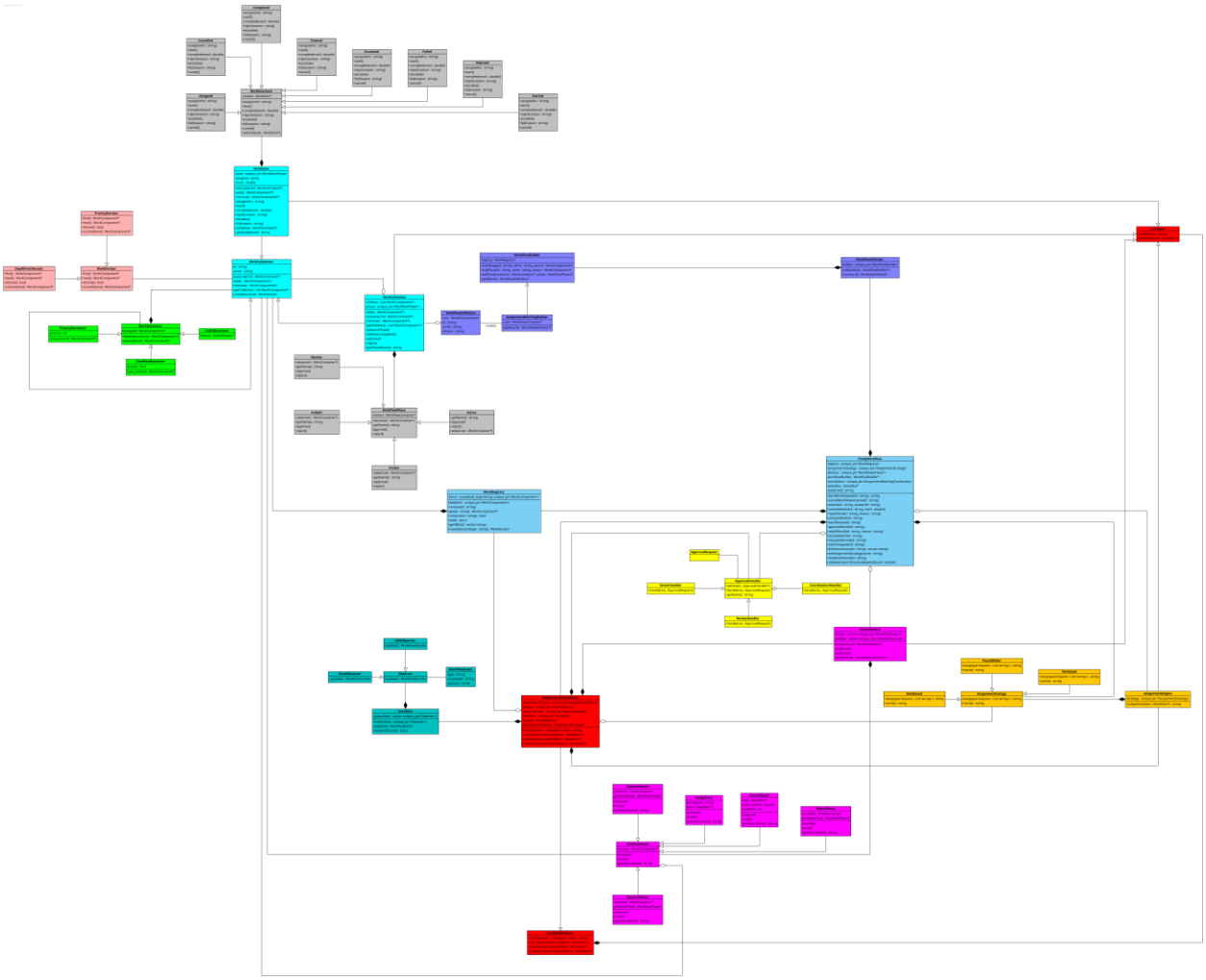
CoordinationHandler(Coordinator marks for example when item gets rejected 3 times)

Façade:

Facade: TividyWorkflow

Subclasses: WorkflowDirector / WorkflowBuilder, WorkContainer and WorkItem, AssignmentEngine, ActionHistory, WorkflowCoordinator, ApprovalHandler, EventBus, Workregistry

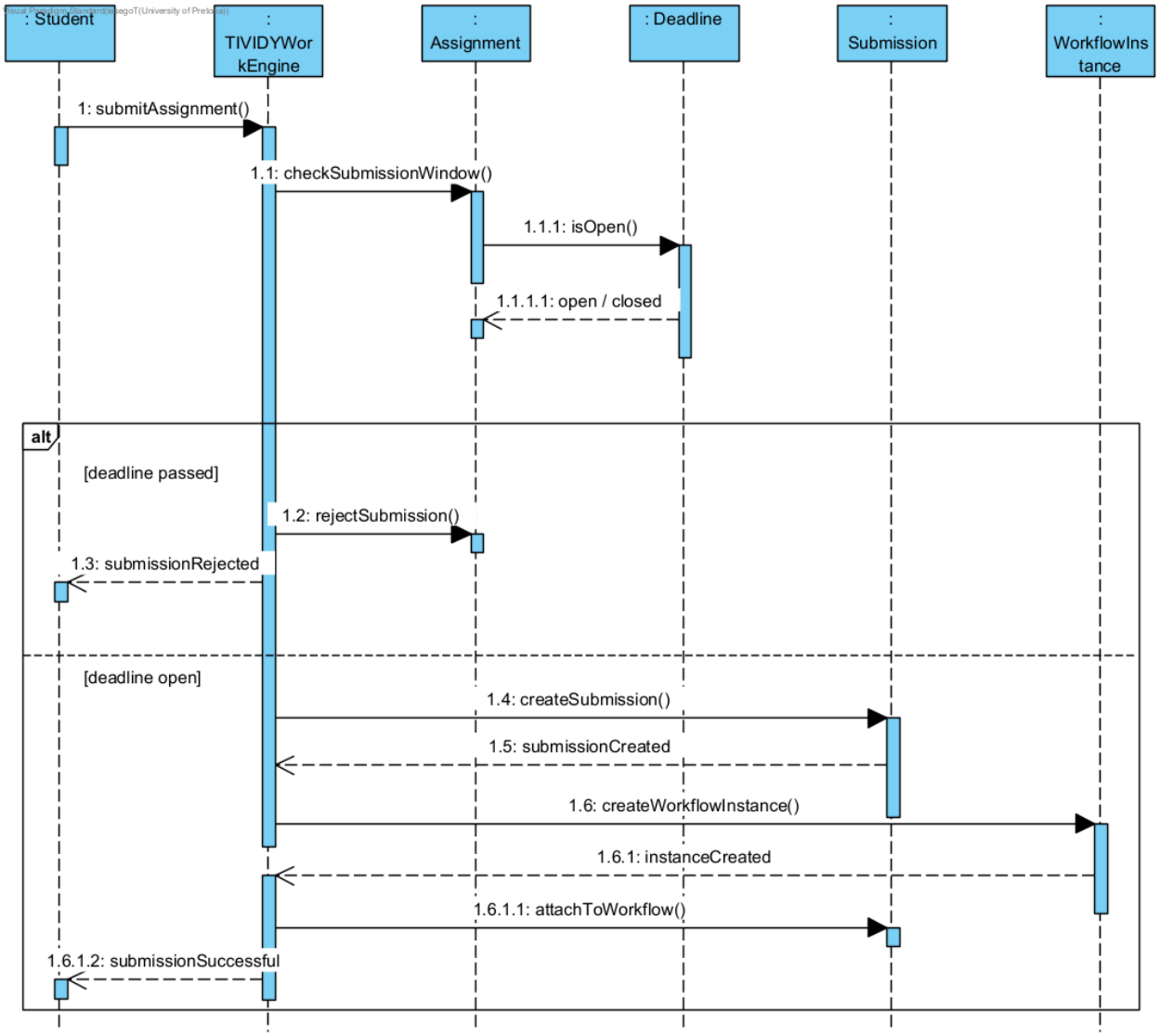
Task 5: UML Class diagram



Task 6: Runtime Behaviour Diagrams

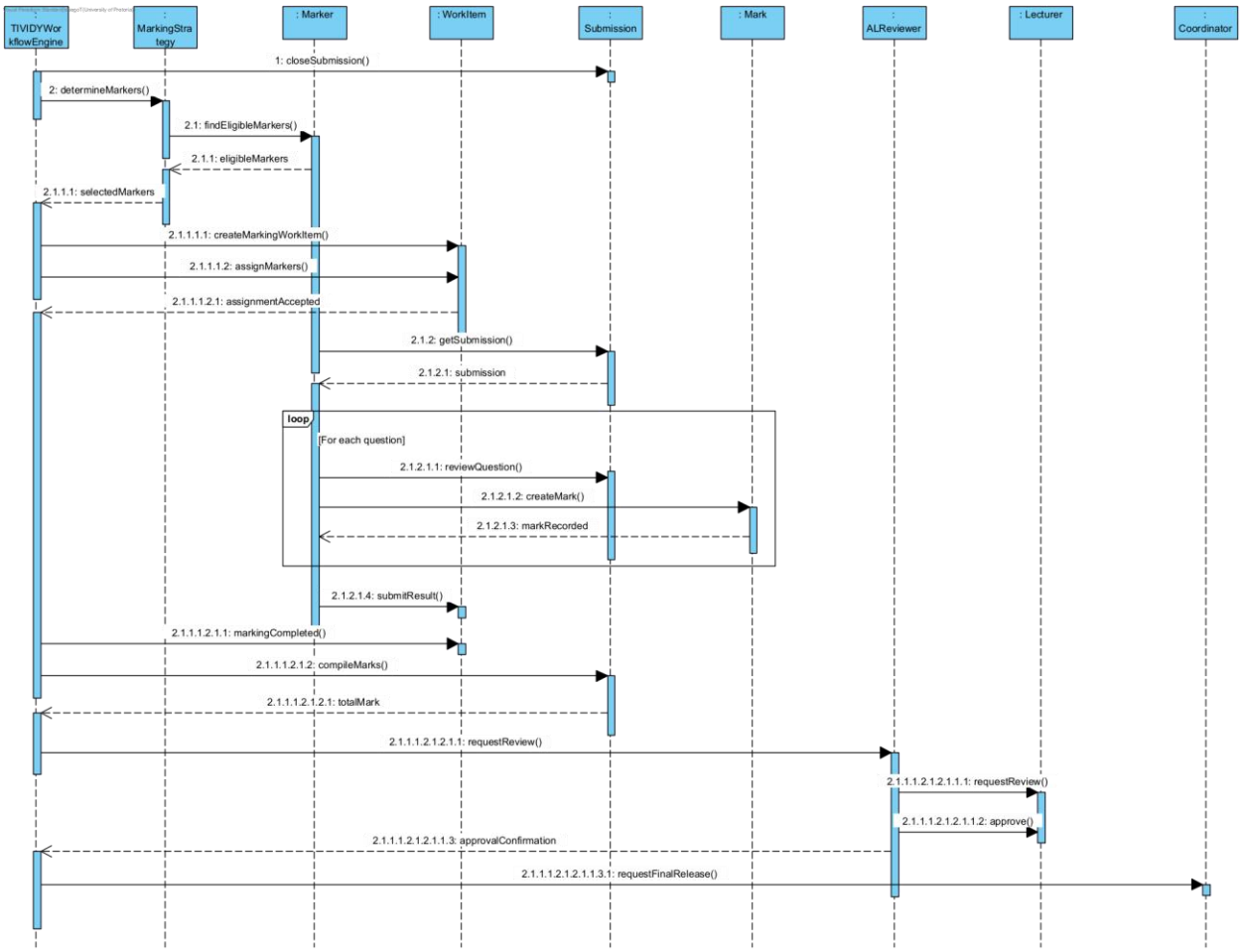
SD1: Assignment Submission

A student submits an assignment through TIVIDY. TIVIDY checks whether the submission deadline is still open. If the deadline has passed, the submission is rejected. If the submission is valid, TIVIDY creates a Submission and associates it with a new Workflow Instance that will manage the marking process.



SD2: Marking, Review and Release

After submissions close, TIVIDY uses a configurable Strategy to determine suitable markers. A marking WorkItem is created and assigned to a marker. The marker evaluates each question and records marks. TIVIDY then compiles the marks and sends them for review. If the review requires adjustment, the work is returned for correction and marking. If the marks are accepted, the workflow progresses through the required approval levels before the final marks are released.



Task 7: Design Decisions and Revisions